

AI Trading Automation API

Signalling Service

This document describes the first AI service exposed through the AI Trading Automation portal: the **Signalling** service. A signalling agent is used to build and operate agentic AI for trading and investment assistance. It monitors a market instrument, analyzes historical data and optional news context, and emits trading opportunities when configured thresholds and risk constraints are met. [file:1]

This document is intended to let consumers use the Signalling service as a black box. The Signalling service supports three operations: **Create**, **Launch**, and **Signal**. Create registers and configures a signalling agent, Launch activates a previously created agent, and Signal returns the current signal output of a launched or available agent. [file:1]

The Create flow for the Signalling service is persistence-first. A Create request is stored in the database so it can later be fetched by the AI server. The AI server then initializes the model, initializes the dataset, trains the model, stores the weights, and updates the agent status in the database. The Launch and Signal operations are exposed as HTTP endpoints. [file:1]

Service scope

The current lifecycle status values are:

- **PENDING**
- **INPROGRESS**
- **FAILED**
- **CREATED**
- **INACTIVE**
- **ACTIVE** [file:1]

Current documented lifecycle order:

```
PENDING -> INPROGRESS -> FAILED | CREATED -> INACTIVE -> ACTIVE
```

The exact semantics of each state should be considered draft until the final implementation contract is frozen. [file:1]

Architecture note

Create configures and builds a signalling agent. It stores the configuration required by the AI server to later fetch the request, build the dataset, train the model, store the weights, and update the agent status. [file:1]

For the Signalling service, Create does not currently use the REST API. Instead, the request is written directly into the database for persistence and later consumption by the AI server. [file:1]

Matching config files

The following request fields must match values defined in configuration JSON files:

- `equity` must match `configs/entities.json`
- `time_frequency` must match `configs/time_frequencies.json`
- `news_resources` entries must match `configs/news_resources.json`

These files are expected to remain relatively stable, but the application should read them dynamically so future updates can be picked up without code changes, for example when targeting a new market or adding a new news provider. [file:1]

Implementation note: The exact contents of `configs/entities.json`, `configs/time_frequencies.json`, and `configs/news_resources.json` should be attached to this document or copied into an appendix so integrators can implement strict validation and ENUM-like constraints where appropriate.

Create

Purpose

Create configures and builds a signalling agent. It stores the configuration required by the AI server to later fetch the request, build the dataset, train the model, store the weights, and update the agent status. [file:1]

Ingestion model

For the Signalling service, Create does not currently use the REST API. Instead, the request is written directly into the database for persistence and later consumption by the AI server. [file:1]

Stored schema

Field	Type	Required	Description
<code>service</code>	string	Yes	Service routing key used by the API gateway and agent manager to dispatch the request to the correct handler. The user does not fill this field manually; it is set automatically by the client SDK or frontend. Current valid value: <code>"signaling"</code> . Future planned values may include <code>"backtesting"</code> , <code>"regime_detection"</code> , and <code>"risk_management"</code> . [file:1]
<code>equity</code>	string	Yes	Financial instrument or entity identifier. Must match <code>configs/entities.json</code> . [file:1]
<code>agent_id</code>	string	Yes	Unique identifier of the agent instance created by the system. Must be non-empty and unique. This field should be used consistently in Create, Launch, and

Field	Type	Required	Description
			Signal. The earlier <code>model_id</code> naming is misleading because Launch and Signal refer to the created agent rather than a model version. [file:1]
<code>time_frequency</code>	string	Yes	Input time resolution such as <code>1MIN</code> , <code>5MIN</code> , <code>1H</code> , <code>1D</code> . Must match <code>configs/time_frequencies.json</code> . [file:1]
<code>observation_horizon</code>	integer	Yes	Number of past time steps used as model input. This is an integer count of steps, not an absolute duration. The actual lookback duration depends on <code>time_frequency</code> . Example: <code>120</code> with <code>1H</code> means 5 days, <code>120</code> with <code>1MIN</code> means 2 hours, and <code>30</code> with <code>1D</code> means about 6 weeks. Static deployment limits are expected but not yet finalized. [file:1]
<code>prediction_horizon</code>	integer	Yes	Number of time steps ahead to forecast. This is an integer count of steps, not an absolute duration. The actual forecast duration depends on <code>time_frequency</code> . Static deployment limits are expected but not yet finalized. [file:1]
<code>change_percentage_threshold</code>	float	No	Minimum forecasted price-change percentage required to trigger a signal. Default: <code>0.02</code> . Intended range: <code>[0.001, 0.5]</code> . Validation rule: reject if <code><= 0</code> or <code>> 1.0</code> . [file:1]
<code>news_observation_horizon</code>	integer	No	Number of past time steps scanned for relevant news. Default: <code>observation_horizon</code> . Valid range: <code>[1, observation_horizon]</code> . Validation rule: must be <code><= observation_horizon</code> . [file:1]
<code>news_retrieval_prompt</code>	string	No	Natural-language query used for news retrieval. Maximum length: <code>500</code> . Default: <code>null</code> . If <code>null</code> and <code>news_resources</code> is provided, the system uses a default prompt per equity. [file:1]
<code>news_resources</code>	array[string]	No	Approved news sources listed in <code>configs/news_resources.json</code> .

Field	Type	Required	Description
			Default: all keys from <code>configs/news_resources.json</code> . Validation rule: reject unknown keys with HTTP 400 and list valid options. [file:1]
<code>confidence_level</code>	float	No	Minimum confidence required to emit a signal. Range: <code>[0.0, 1.0]</code> . Default: <code>0.70</code> . Validation rule: reject if outside <code>[0.0, 1.0]</code> . [file:1]
<code>signal_frequency</code>	integer	Yes	Frequency, in time steps, at which the model evaluates whether to generate a signal. [file:1]
<code>status</code>	string	Yes	Lifecycle state of the agent. Valid values: <code>PENDING</code> , <code>INPROGRESS</code> , <code>FAILED</code> , <code>CREATED</code> , <code>INACTIVE</code> , <code>ACTIVE</code> . [file:1]

Optional input policy

For every field where `Required = No`, the user should still be allowed to provide it, but the field is optional rather than mandatory. If the user does not provide it, the system should apply the documented default behavior. [file:1]

Example record

```
{
  "service": "signaling",
  "equity": "AAPL",
  "agent_id": "signal-agent-aapl-001",
  "time_frequency": "1H",
  "observation_horizon": 120,
  "prediction_horizon": 8,
  "change_percentage_threshold": 0.02,
  "news_observation_horizon": 24,
  "news_retrieval_prompt": "Find recent company-specific and macroeconomic news with potential impact on AAPL.",
  "news_resources": ["reuters", "bloomberg"],
  "confidence_level": 0.8,
  "signal_frequency": 1,
  "status": "PENDING"
}
```

Launch

Endpoint

```
POST /launch
```

Note: the current path is documented exactly as provided: `/launch`. [file:1]

Purpose

Launch activates an existing signalling agent so it can start generating live signals. The request uses the submitted service and agent identifier to fetch the agent from the database. [file:1]

Request body

Field	Type	Required	Description
<code>service</code>	string	Yes	Service routing key used to resolve the target service. Current valid value: <code>"signaling"</code> . The user does not fill this manually; it is set automatically by the client SDK or frontend. [file:1]
<code>agent_id</code>	string	Yes	Unique identifier of the agent to launch. This should refer to the identifier created during Create. [file:1]

Example request

```
{
  "service": "signaling",
  "agent_id": "signal-agent-aapl-001"
}
```

Expected behavior

After launch, the system should set up a mechanism to automatically send live signals generated by the agent. The transport for live delivery is still to be finalized, but the signal payload structure is expected to remain the same as the standard Signal response. [file:1]

Signal

Endpoint

```
POST /signal
```

Purpose

Signal returns the current signal output for an agent identified by `service` and `agent_id`. The endpoint fetches the agent from the database and returns the generated trading signal, confidence values, and risk-sizing details when available. [file:1]

Request body

Field	Type	Required	Description
<code>service</code>	string	Yes	Service routing key used to resolve the target service. Current valid value: <code>"signaling"</code> . The user does not fill this manually; it is set automatically by the client SDK or frontend. [file:1]
<code>agent_id</code>	string	Yes	Unique identifier of the agent whose signal is requested. [file:1]

Example request

```
{
  "service": "signaling",
  "agent_id": "signal-agent-aapl-001"
}
```

Response schema

Field	Type	Description
<code>estimated_action</code>	string (ENUM)	Predicted direction recommended by the model. Possible values: <code>"buy"</code> <code>"sell"</code> <code>"hold"</code> . Always lowercase. Maps directly to the winning class in probabilities. Not nullable. [file:1]
<code>signal</code>	string (ENUM)	Final signal label exposed to the user after applying <code>confidence_level</code> and <code>change_percentage_threshold</code> filters. Possible values: <code>"BUY"</code> <code>"SELL"</code> <code>"HOLD"</code> . Always uppercase. Not nullable. [file:1]
<code>probability</code>	float	Confidence score of the winning class. Range: <code>[0.0, 1.0]</code> . Rounded to 4 decimal places. Corresponds to <code>max(probabilities.buy, probabilities.sell, probabilities.hold)</code> . Not nullable. [file:1]
<code>probabilities.sell</code>	float	Raw softmax probability assigned to the sell class. Range: <code>[0.0, 1.0]</code> . Rounded to 4 decimal places. Not nullable. [file:1]
<code>probabilities.hold</code>	float	Raw softmax probability assigned to the hold class. Range: <code>[0.0, 1.0]</code> . Rounded to 4 decimal places. Not nullable. [file:1]
<code>probabilities.buy</code>	float	Raw softmax probability assigned to the buy class. Range: <code>[0.0, 1.0]</code> . Rounded to 4 decimal places. Not nullable. [file:1]
<code>volume</code>	float null	Suggested number of units or shares to trade, computed by the risk sizing engine. Range when present: <code>> 0.0</code> . Precision: up to 4 decimal places. Nullable. <code>null</code> when no risk sizing method is configured or when the risk engine returns zero size. [file:1]

Field	Type	Description
notional	float null	Suggested notional exposure in account currency. Range when present: > 0.0. Rounded to 2 decimal places. Nullable. [file:1]
stop_loss_price	float null	Suggested stop-loss price level. Range when present: > 0.0. Up to 4 decimal places. For a buy signal, stop_loss_price < entry_price. For a sell signal, stop_loss_price > entry_price. Nullable. [file:1]
risk_amount	float null	Absolute capital at risk for this trade in account currency. Range when present: > 0.0. Rounded to 2 decimal places. Nullable. [file:1]
sizing_method	string null	Risk sizing method used by the engine. Possible values when present: "fixed_fractional" "kelly" "cvar" null. Nullable. [file:1]
warnings	array[string]	Warning list returned by the risk layer or signal generation logic. Never null; default is []. Allowed values: "CONCENTRATION_RISK", "RISK_PCT_ZERO", "SMALL_TAIL_SAMPLE_WARNING", "NO_EDGE", "LOW_CONFIDENCE", "MISSING_ATR", "INSUFFICIENT_CAPITAL". [file:1]

Response invariants

The following response rules apply:

- probabilities.buy + probabilities.sell + probabilities.hold must sum to 1.0 within floating-point tolerance. [file:1]
- signal may be "HOLD" even when estimated_action is "buy" or "sell" if post-filter logic suppresses the action due to low confidence or threshold constraints. [file:1]
- warnings must always be present as an array, even when empty. [file:1]

Example response

```
{
  "estimated_action": "buy",
  "signal": "BUY",
  "probability": 0.8421,
  "probabilities": {
    "sell": 0.0712,
    "hold": 0.0867,
    "buy": 0.8421
  },
  "volume": 12.5,
  "notional": 2450.75,
  "stop_loss_price": 192.45,
  "risk_amount": 120.0,
  "sizing_method": "fixed_fractional",
```

```
"warnings": []  
}
```

Live signals

After an agent is launched, live signals must be sent automatically by the agent. The delivery approach is not yet finalized in this draft. Regardless of transport, the live signal payload should use the same schema as the `/signal` response. [file:1]

Consumer guidance

To use the Signalling service as a black box, a client should follow this sequence:

1. Persist a valid Create request in the database. [file:1]
2. Wait until the agent is successfully built and available. [file:1]
3. Launch the agent with `POST /launch`. [file:1]
4. Fetch signals with `POST /signal` or consume the future live-signal delivery channel. [file:1]

Draft note

This contract is still under development. Some fields, endpoints, behaviors, lifecycle rules, and validation limits may change before the final production version is published. [file:1]